

Curso de Especialización

Ciberseguridad en las TIC

Ejercicio-DEVSECOPS-7

Objetivos

- Analizar por fases DevSecOps dónde hay incumplimientos.
- Identificar en qué fase o fases se incumple la seguridad.
- Descubrir qué riesgo concreto existe.
- Saber cómo debería corregirse en cada fase.

Resumen

Autora: Marina del Pilar López Oliva

Marzo 2026

IES Camp de Morvedre

Obra publicada con [Licencia Creative Commons Reconocimiento Compartir igual 4.0](https://creativecommons.org/licenses/by-sa/4.0/)



Revisión

Revisión	Fecha	Descripción
1.0	24/03/2026	Realización del ejercicio.

Índice de Contenidos

Analiza por fases DevSecOps dónde hay incumplimientos.....	4
Plan.....	4
Code.....	4
Build.....	5
Test.....	6
Release.....	6
Deploy.....	7
Monitor.....	8

Analiza por fases DevSecOps dónde hay incumplimientos

Plan

Define requisitos de seguridad como las tecnologías elegidas, requisitos funcionales y requisitos de seguridad definidos. Se menciona que se utilizará para el backend Python + Flask, se usará una base de datos Mongo DB, se usará autenticación por token personalizado y el frontend utilizará react. No se mencionan los requisitos de las versiones a utilizar.

Se especifica que los pacientes pueden registrarse, ver su historia y pedir citas. Además se recalca que se debe utilizar HTTPS. También menciona que se tendrán que almacenar logs.

Por esta parte el plan está bastante bien formado, ya que especifica varios requisitos de seguridad. Sin embargo, no se menciona nada de un cifrado de datos médicos sensibles, el cumplimiento de normativa, los controles de accesos por roles, la protección frente a OWASP Top 10 ni una rotación de tokens. Tampoco la caducidad de los tokens de autenticación ni se analizan las amenazas. Tampoco se planifica validaciones de entradas ni políticas de contraseña.

Falla el plan porque no tiene al OWASP Top 10.

Qué cambiaría:

- Versiones mínimas y una política de contraseña.
- Análisis de amenazas.
- Requisitos de cumplimiento normativo.
- Pensar el plan en OWASP Top 10.
- Añadir un control de accesos detallado.
- Planificar la validación de entradas y utilización de parámetros.

Code

El código presenta varias vulnerabilidades visibles y claras:

- Falta validación de entradas en los campos que introduce el usuario:
`username = request.json["username"] password = request.json["password"]`
- Puede haber SQL Injection si el nombre o la contraseña no son strings.
- La contraseña se guarda en texto plano.
- El token no expira ni está firmado, está en el código.
- No tiene rate limit para los login.

- Los datos se concatenan en la base de datos, lo que puede exponer los datos sensibles del historial.
- Datos sensibles sin cifrar, también guardados en texto plano.

Falla el code porque tiene vulnerabilidades como SQL Injection, contraseñas en texto plano, token en código, etc. Y además no tiene control de errores.

Qué cambiaría:

- Usar consultas parametrizadas con el parámetro "?".
- Hashear las contraseñas con alguna librería.
- Implementar gestión de sesiones.
- Añadir rate limiting para evitar ataques de fuerza bruta.
- Usar tokens con expiración y seguros.
- Validar las entradas y rechazar payloads no esperados.
- Cifrar la información sensible.
- Devolver solo los datos necesarios de la base de datos.
- Añadir una gestión de roles.

Build

Se realizó un informe SCA que detectó vulnerabilidades conocidas en Flask 0.12, además detectó que el pymongo estaba desactualizado y que no aplican actualizaciones automáticas.

Sin embargo, no se ejecutó SAST ni un escaneo de vulnerabilidades conocidas cuando se dio el caso. Tampoco se revisaron las dependencias vulnerables ni se utilizó un ci/cd seguro. No se realizó un escaneo de integridad ni se validaron las claves en código.

En un pipeline DevSecOps BUILD debería fallar cuando se detectan dependencias con CVE de cierta criticidad, por lo tanto falla por uso de dependencias vulnerables.

Qué cambiaría:

- SAST para detectar todas las vulnerabilidades conocidas.
- Comprobar las versiones utilizadas y actualizarlas.
- Ejecutar un checklist automático.
- Configurar actualizaciones automáticas.

- Realizar un chequeo que bloquee la continuación de la aplicación cuando detecte dependencias con CVE de cierta criticidad.

Test

Se realizó una prueba de registro de usuario y de login correcto para ver el correcto funcionamiento de la función de login. También se hizo una prueba para comprobar la función de crear citas.

Sin embargo, no se realizó una prueba para ver si se podía acceder a datos de otros usuarios, lo que sería un problema para la confidencialidad. Tampoco se realizaron pruebas de manipulación de tokens ni de fuga de datos sensibles. No se comprobó si se podría hacer una enumeración de usuarios entrando en uno ni ataques de NoSQL Injection.

No se probaron inputs maliciosos, ni se interceptó tráfico con burp para probar un ataque malicioso. Tampoco se realizó un bypass de login ni se probaron a manipular parámetros. No probaron problemas con la autorización ni se realizó un pentesting. No se hicieron pruebas de seguridad.

El test falla actualmente, habría que integrar SAST, SCA y DAST idealmente.

Qué cambiaría:

- Ejecutar un análisis DAST antes del release.
- Probar las vulnerabilidades encontradas anteriormente y algunas básicas.
- Establecer criterios de aceptación de seguridad para no pasar a release.
- Realizar pruebas para ver acceso a datos de otros usuarios.
- Probar la manipulación de tokens y parámetros, o de ataques reales que pueda sufrir la aplicación.
- Comprobar si existe fuga de datos o enumeración de usuarios.

Release

Se publicó sin una revisión final, saltándose la aprobación y la validación del checklist de seguridad. Se observó que el archivo .env incluido en el repositorio dejaba a la vista la url hacia el root y la contraseña secreta. Dando a entender que cualquiera podría entrar como root. Además, se observa que hay commits antiguos que contienen credenciales aún en uso y no hay control de acceso al repositorio. También se observa que no usan firmas de commits.

Las credenciales se dejaron por defecto y se publicó con vulnerabilidades conocidas sin corregirlas. Los resultados del SAST no permitirían pasar esta versión, por lo que debería haberse bloqueado el release.

El release falla porque no se usan los resultados de seguridad.

Qué cambiaría:

- Implementar un security gate automático.
- Exigir aprobación explícita del responsable de seguridad antes del despliegue.
- Gestionar las versiones y hacer un checklist de seguridad.
- Borrar commits antiguos con credenciales reales.
- Controlar el acceso al repositorio.
- Eliminar el archivo .env del repositorio público.
- Usar firmas de commit.

Deploy

Está ejecutando con la el servidor de desarrollo de flask flask run. El servidor tiene el modo debug activado. La base de datos está expuesta en el servidor, tiene acceso público, lo que aumenta el riesgo a la integridad y disponibilidad. Puerto 27017 abierto a toda la red. No hay HTTPS real solo se ha configurado en el código pero no se ha expuesto. Sin firewall configurado para restricciones de acceso, sin un reverse proxy como nginx, sin sistema de logs centralizados ni limitación de peticiones.

En deploy debería usar servidor de producción como Gunicorn.

El deploy falla porque el servidor está utilizando el desarrollo, el debug está en producción y está expuesto a la red.

Qué cambiaría:

- Usar un servidor de desarrollo que no sea el de flask en producción.
- Gestionar secretos con variables de entorno.
- Exigir la utilización de https.
- Restringir el acceso sólo a la red interesada.
- Configurar un firewall robusto.
- Utilizar un reverse proxy.
- Configurar un sistema de logs centralizados.

- Establecer limitación de las peticiones.

Monitor

Servidor de desarrollo de flaks funcionando y en modo debug. Las contraseñas se ven en texto plano. No hay rate limit para el login. Se observan tres login seguidos con tres contraseñas distintas, lo que podría ser un ataque de fuerza bruta. Los tokens también aparecen en los logs en texto plano sin cifrar. Sin alertas disponibles por patrones de inyección o ataques de fuerza bruta. Uso continuado de debug, sin centralización de logs y dashboard de seguridad. Parece que haya un ataque de fuerza bruta para entrar en el usuario "juan". Aparece que el acceso fue concebido pero hubo un fallo de conexión a la base de datos, lo que podría deberse a un ataque DoS a la base de datos ya que es completamente pública.

El monitor existe algo de logging, pero no hay monitorización ni alertas, por lo tanto falla, aunque los datos están ahí.

Qué cambiaría.:

- Configurar alertas automáticas y respetar los cambios anteriores.
- No poner en los logs datos sensibles.
- Definir un claro plan de respuestas a incidentes.
- Integrar un SIEM.